Advanced disk management (RAID)

What is RAID ?

RAID stands for “Redundant Array of Inexpensive Disks”.

The basic idea behind RAID is to combine multiple small, inexpensive disk drives into an array which yields performance exceeding that of one large and expensive drive. This array of drives will appear to the computer as a single logical storage unit or drive.

Some of the concepts used in this chapter are important factors in deciding which level of RAID to use in a particular situation. Parity is a concept used to add redundancy to storage. A parity bit is a binary digit which is added to ensure that the number of bits with value of one in a given set of bits is always even or odd. By using part of the capacity of the RAID for storing parity bits in a clever way, single disk failures can happen without data-loss since the missing bit can be recalculated using the parity bit.

Redundancy

Redundancy is a feature that allows an array to survive a disk failure. Not all RAID levels support this feature. In fact, although the term RAID is used to describe certain types of non-redundant arrays, these arrays are not, in fact, RAID because they do not support any data redundancy.

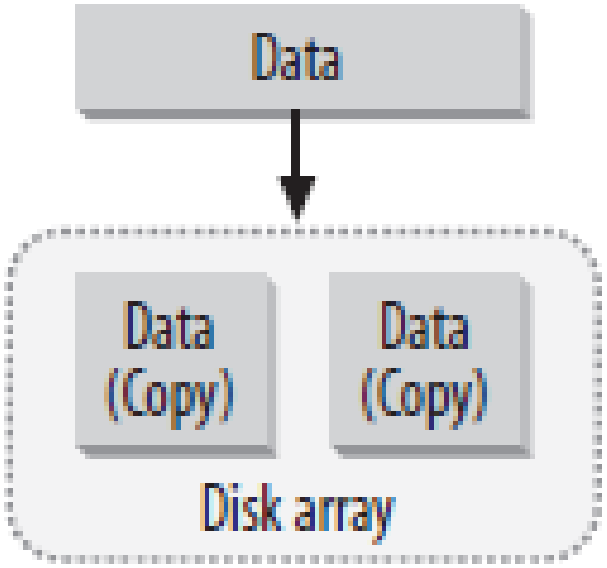
Note:

Despite its redundant capabilities, RAID should never be used as a replacement for reliable backups. RAID does not protect your data in

the event of a fire, natural disaster, or user error.

Mirroring (Continued)

Mirroring replicates data onto every disk in the array. Each member disk contains the same data and has an equal role in the array. In the event of a disk failure, data can be read from the remaining disks.



Mirroring

Improved read performance is a by-product of disk mirroring. When the array is operating normally, meaning that no disks have failed, data can be read in parallel from each disk in the mirror.

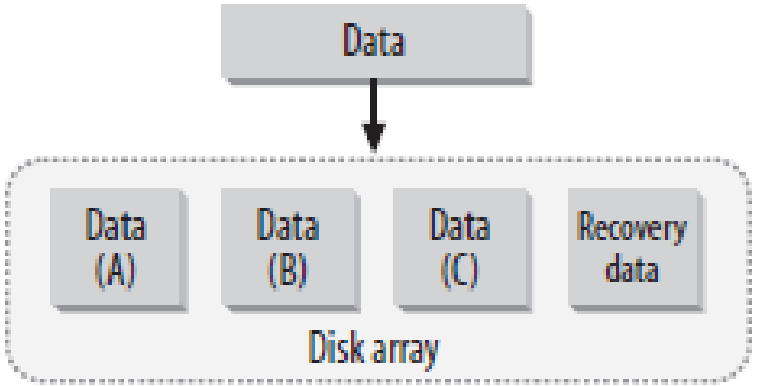
The result is that reads can yield a linear performance based on the number of disks in the array. A two-disk mirror could yield read speeds up to two times that of a single disk. However, in practice, you probably won't see a read performance increase that's quite this dramatic. That's because many other factors, including filesystem performance and data distribution, also affect throughput.

But you can still expect read performance that's better than that of a single disk. Unfortunately, mirroring also means that data must be written twice—once to each disk in the array. The result is slightly slower write performance, compared to that of a single disk or no mirroring array.

Parity

Parity algorithms are the other method of redundancy. When data is written to an array, recovery information is written onto a separate disk.

If a drive fails, the original data can be reconstructed from the parity information and the remaining data.



Degraded

Degraded describes an array that supports redundancy, but has one or more failed disks. The array is still operational, but its reliability and, in some cases, its performance, is diminished. When an array is in degraded mode, an additional disk failure usually indicates data loss.

Reconstruction, resynchronization, and recovery

When a failed disk from a degraded array is replaced, a recovery process begins. The terms *reconstruction*, *resynchronization*, *recovery*, and *rebuild* are often used interchangeably to describe this recovery process. During recovery, data is either copied verbatim to the new disk (if mirroring was used) or reconstructed using the parity information provided by the remaining disks (if parity was used).

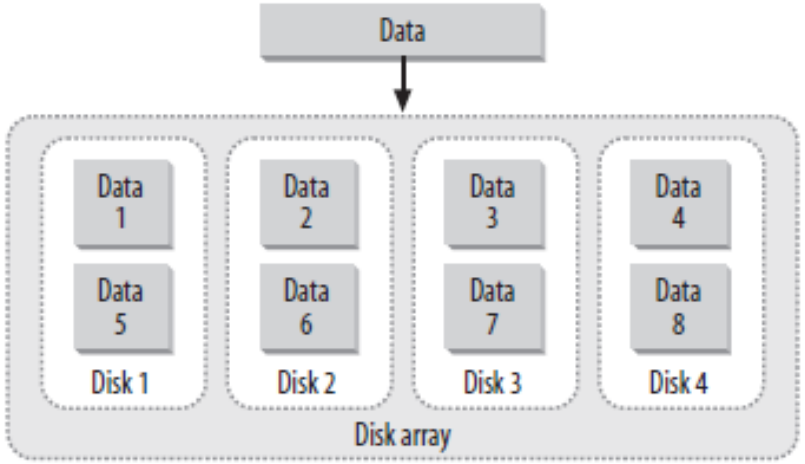
The recovery process usually puts an additional strain on system resources. Recovery can be automated by both hardware and software, provided that enough hardware (disks) is available to repair an array without user intervention.

Whenever a new redundant array is created, an initial recovery process is performed. This process ensures that all disks are synchronized. It is part of normal RAID operations and does not indicate any hardware or software errors.

Striping

Striping is a method by which data is spread across multiple disks. A fixed amount of data is written to each disk. The first disk in the array is not reused until an equal amount of data is written to each of the other disks in the array. This results in improved read and write performance, because data is written to more than one drive at a time.

Some arrays that store data in stripes also support redundancy through disk parity. RAID-0 defines a striped array without redundancy, resulting in extremely fast read and write performance, but no method for surviving a disk failure. Not all types of arrays support striping.



Linear Mode

This isn't technically RAID, but it's handled by Linux's RAID subsystem.

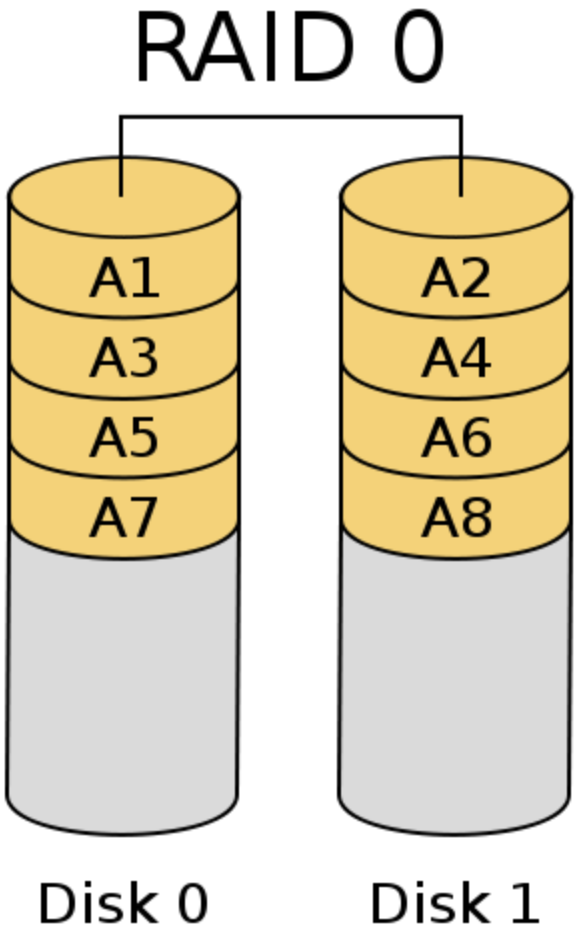
In linear mode, devices are combined together by appending one device's space to another's.

Linear mode provides neither speed nor reliability benefits, but it can be a quick way to combine disk devices if you need to create a very large filesystem. The main advantage of linear mode is that you can combine partitions of unequal size without losing storage space; other forms of RAID require equal - sized underlying partitions and ignore some of the space if they're fed unequal - sized partitions.

RAID 0 (Striping)

RAID level 0, often called “striping”, is a performance-oriented striped data mapping technique. This means the data being written to the array is broken down into strips and written across the member disks of the array.

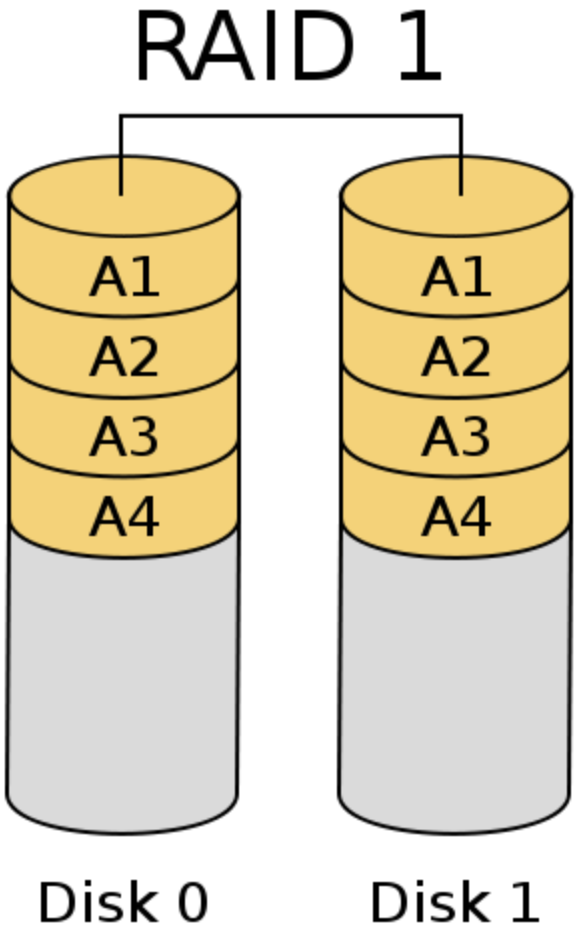
This allows high I/O performance at low inherent cost but provides no redundancy. Storage capacity of the array is equal to the sum of the capacity of the member disks.



RAID 1 (Mirroring)

RAID level 1, or “mirroring”, has been used longer than any other form of RAID. Level 1 provides redundancy by writing identical data to each member disk of the array, leaving a mirrored copy on each disk. Mirroring remains popular due to its simplicity and high level of data availability.

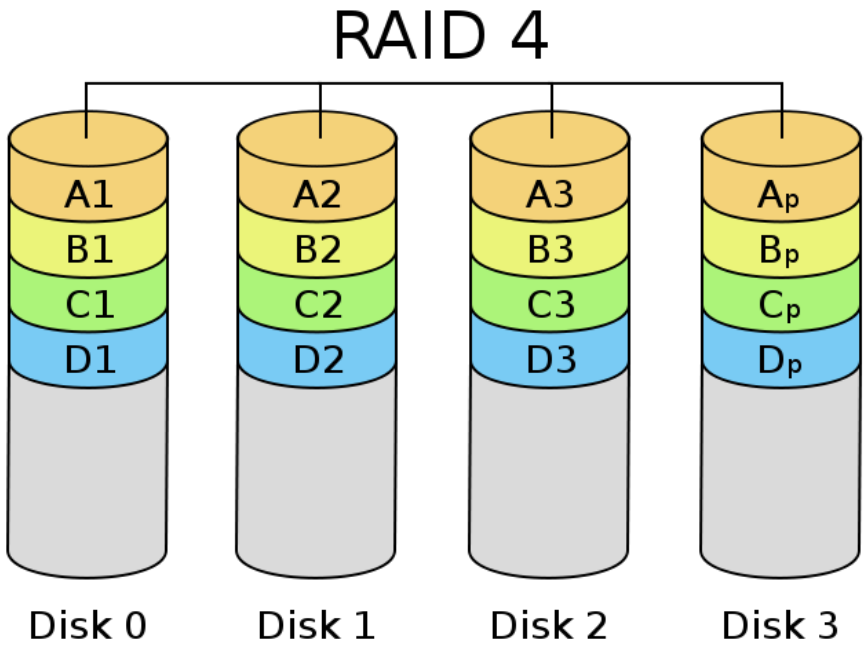
Level 1 operates with two or more disks that may use parallel access for high data-transfer rates when reading, but more commonly operate independently to provide high I/O transaction rates. Level 1 provides very good data reliability and improves performance for read-intensive applications but at a relatively high cost. Array capacity is equal to the capacity of the smallest member disk.



RAID-4: Dedicated Parity

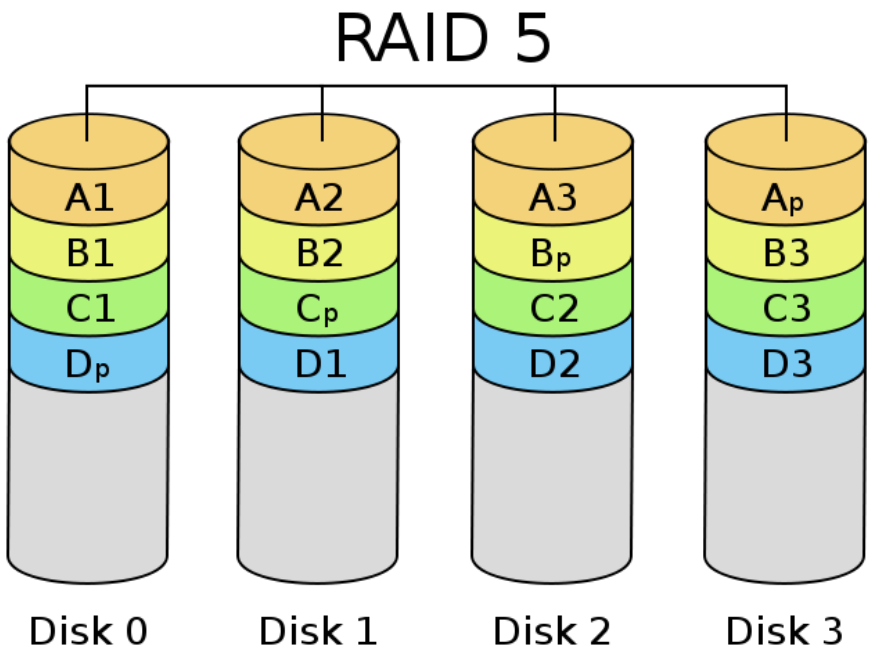
Higher levels of RAID attempt to gain the benefits of both RAID 0 and RAID 1. In RAID 4, data are striped in a manner similar to RAID 0; but one drive is dedicated to holding checksum data. If any one disk fails, the checksum data can be used to regenerate the lost data.

The checksum drive does not contribute to the overall amount of data stored; essentially, if you have n identically sized disks, they can store the same amount of data as n – 1 disks of the same size in a non-RAID or RAID 0 configuration. As a practical matter, you need at least three identically sized disks to implement a RAID 4 array.



RAID-5: Distributed Parity

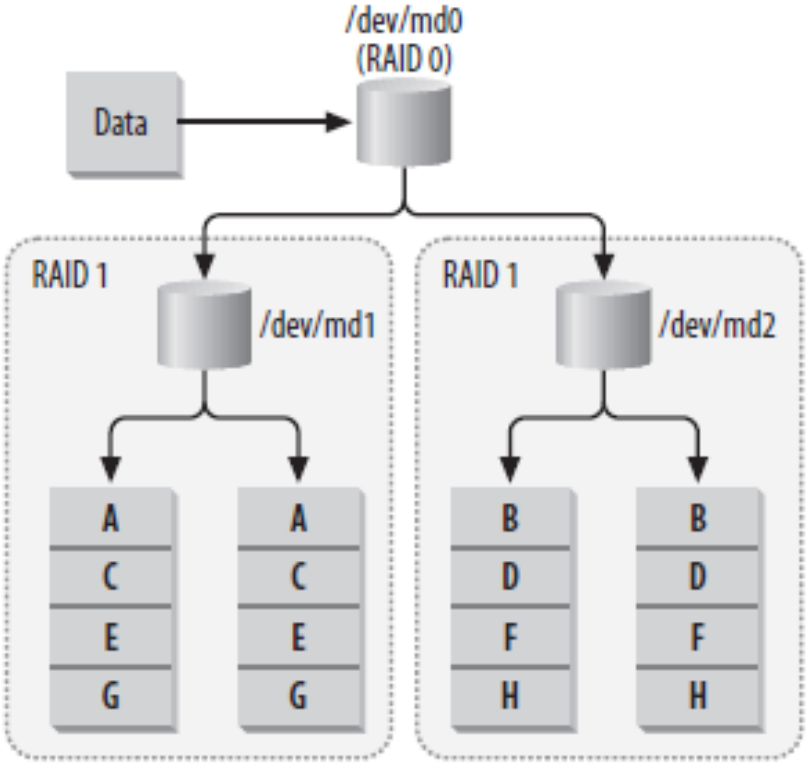
This form of RAID works just like RAID 4, except that there's no dedicated checksum drive; instead, the checksum data are interleaved on all the disks in the array. RAID 5's size computations are the same as those for RAID 4; you need a minimum of three disks, and n disks hold n – 1 disks worth of data.



RAID 10

A combination of RAID 1 with RAID 0, referred to as RAID 1 + 0 or RAID 10, provides benefits similar to those of RAID 4 or RAID 5.

Linux provides explicit support for this combination to simplify configuration.



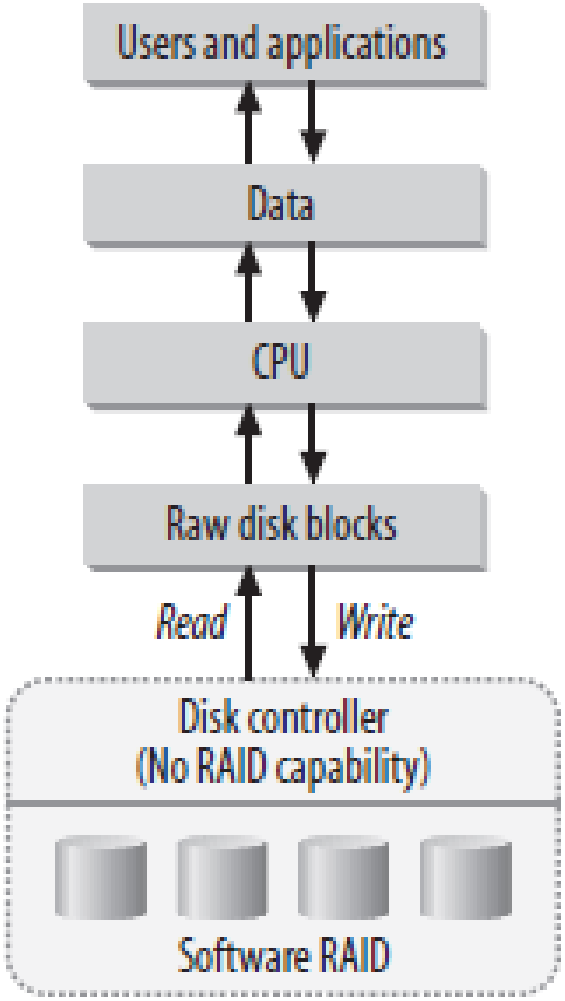
Hardware or Software?

RAID, like many other computer technologies, is divided into two camps: hardware and software. Software RAID uses the computer's CPU to perform RAID operations and is implemented in the kernel.

Hardware RAID uses specialized processors, usually found on disk controllers, to perform array management functions. The choice between software and hardware is the first decision you need to make.

Software (Kernel-Managed) RAID

Software RAID means that an array is managed by the kernel, rather than by specialized hardware. The kernel keeps track of how to organize data on many disks while presenting only a single virtual device to applications. This virtual device works just like any normal fixed disk.

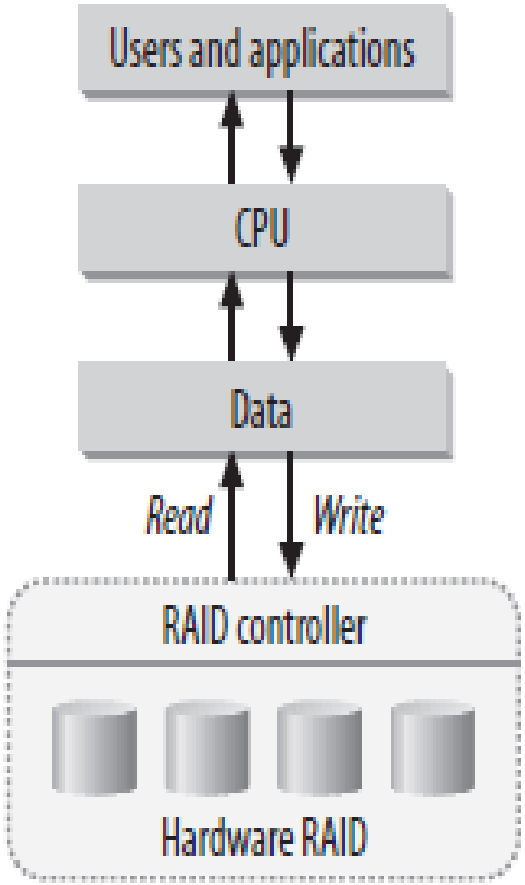


Hardware RAID

Large-scale and expensive hardware RAID solutions are typically faster than software solutions and don't require additional CPU overhead to manage arrays.

Some disk controllers internally support RAID and can manage disks without the help of the CPU. These RAID cards handle all array functions and present the array as a standard block device to Linux.

Hardware RAID cards usually contain an onboard BIOS that provides the management tools for configuring and maintaining arrays. Software packages that run at the OS level are usually provided as a means of post-installation array management. This allows administrators to maintain RAID devices without rebooting the system.



RAID Case Studies

- **What Should I Choose?**
- Choosing an architecture can be extremely difficult. Trying to connect a specific
- technology to a specific application is one of the hardest tasks that system administrators face. Below are some examples of where RAID is useful in the real world.

Case 1: HTTP Image Server

Because RAID-1 supports parallel reads, it makes a great HTTP image server. Companies that sell products online and provide product photos to web surfers could use RAID-1 to serve images. Images are static content, and in this scenario, they will likely be read quite a bit more than they will be written. Although new product photos are frequently added, they are written to disk only once by a web developer, whereas they are viewed thousands of times by potential customers.

Parallel read performance on RAID-1 helps facilitate the large number of hits, and the write performance loss with RAID-1 is largely irrelevant because writes are infrequent in this case.

The redundancy aspect of RAID-1 also ensures that downtime is minimal in the event of a disk failure, although parallel read performance will be temporarily lost until the drive can be replaced. Using a hot-spare, of course, ensures that performance is affected for only a brief time.

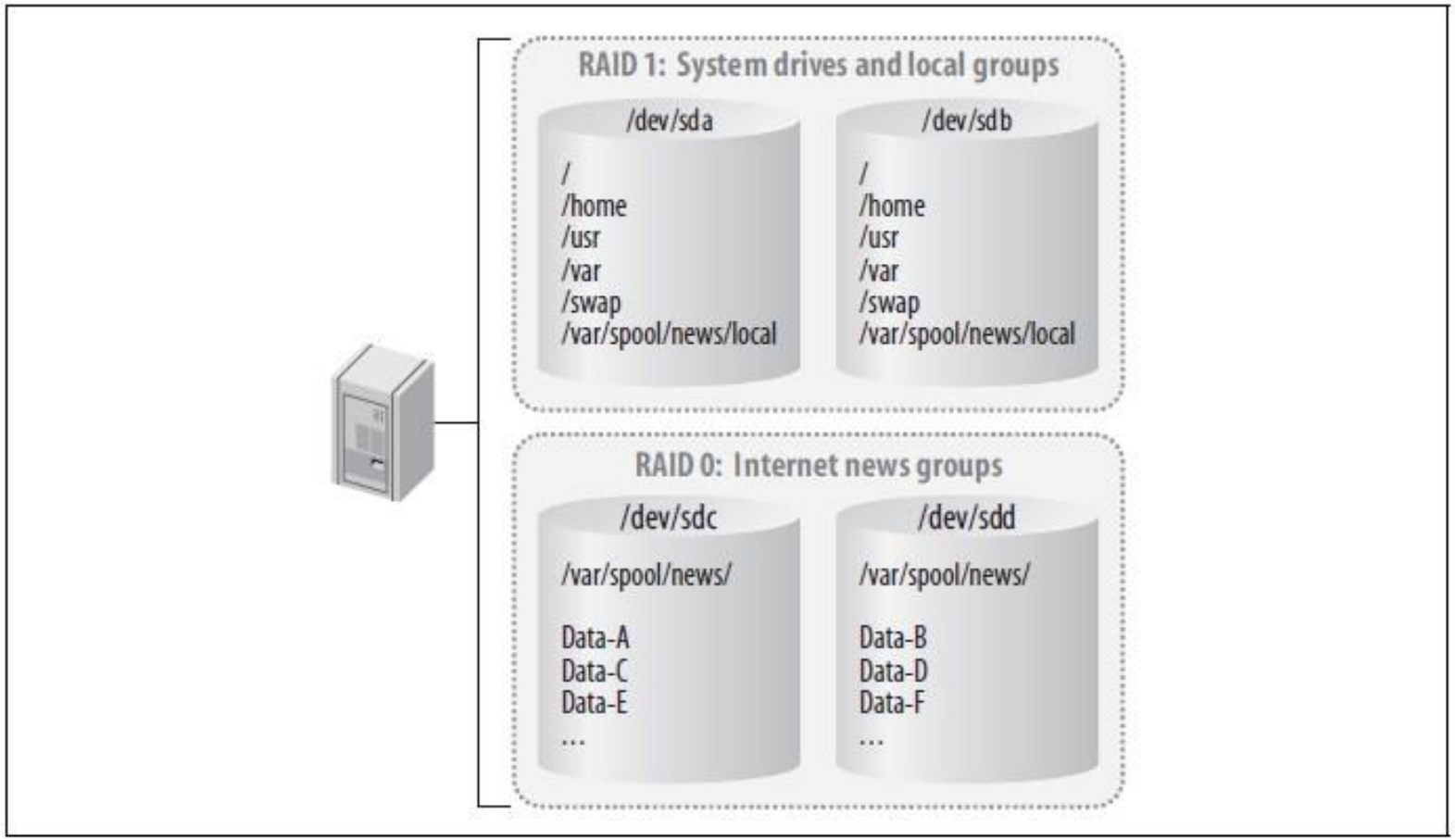
Case 2: Usenet News

Striped arrays are clearly the best candidate for Internet news servers. Extremely fast read and write times are required to keep up with the enormous streams of data that a typical full-feed news server experiences. In many cases, the data on a news partition is inconsequential. Lost articles are frequent, even in normally operating feeds, and complete data loss usually means that only a few days' articles are lost.

Administrators could configure a single news server with both a striped array and mirrored array. The striped array could house newsgroupsthat are of no consequence and could easily withstand a day's worth of article loss without users complaining. Newsgroups that are read frequently, as well as local groups and system partitions, could be housed on the RAID-1 array. This would

make the machine redundant in case of a disk failure.

A Usenet news server with both a striped and mirror array

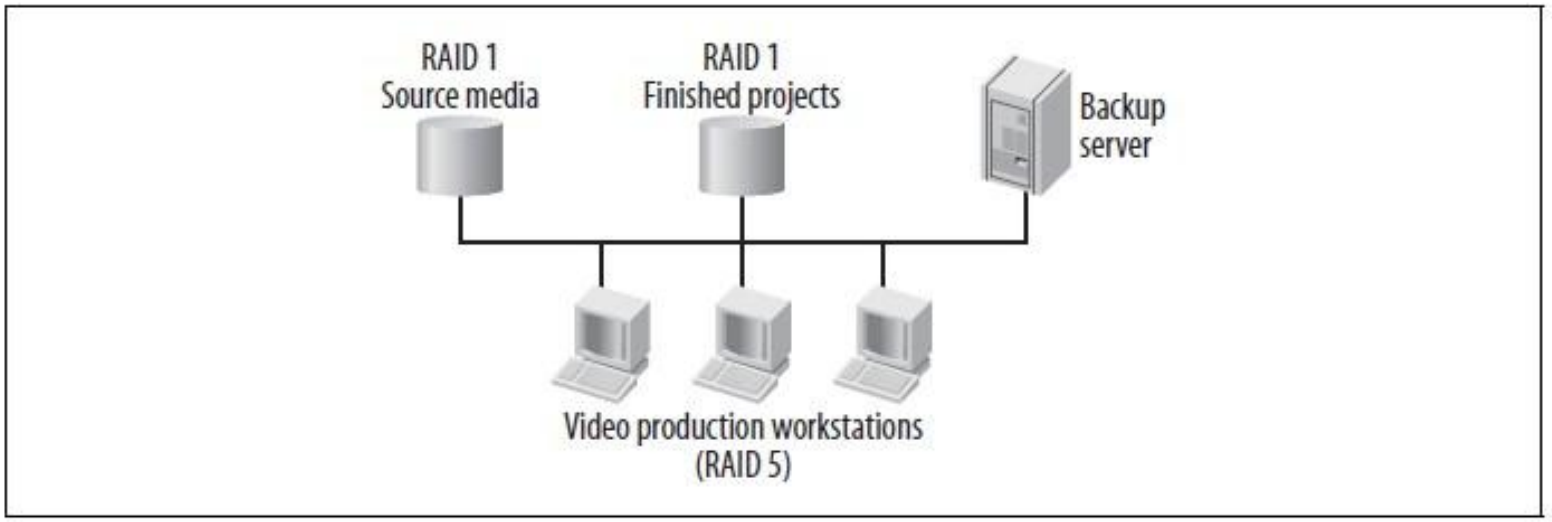


Case 3: The Acme Motion Picture Company

- Film production workstations would benefit greatly from RAID-5. While RAID-0
- might seem like a good choice because of its fast performance, losing a work inprogress might set work back by days, or even weeks. By using RAID-5, editors are able to achieve redundancy and see an improvement in performance.
- Likewise, RAID-1 might seem like a good choice because it offers redundancy without much of a performance hit during disk failures.

Case 4: Video on Demand

This scenario offers the same considerations as Case 1, the site serving images. RAID-1, with multiple member disks, offers great read performance. Since writes aren't very frequent when working with video on demand, the write performance hit is okay.



Workstations with RAID-5 arrays edit films while retrieving source films from a RAID-1 array. Finished products are sent to another RAID-1 array.

Hot-Spares

some RAID levels can replace a failed drive with a new drive without user intervention. This functionality, known as hot-spares, is built into every hardware RAID controller and standalone array. It is also part of the Linux kernel. If you have hardware that supports hot-spares, then you can identify some extra disks to act as spares when a drive failure occurs.

Once an array experiences a disk failure, and consequently enters into degraded mode, a hot-spare can automatically be introduced into the array. This makes the job of the administrator much easier, because the array immediately resumes normal operation, allowing the administrator to replace failed drives when convenient.

In addition, having hot-spares decreases the chance that a second drive will fail and cause data loss.

Working with Software RAID

The raidtools package, also maintained by Ingo Molnar, provides a set of utilities for creating and managing software arrays. raidtools has been the standard software RAID management package for Linux since the inception of the software RAID driver.

It relies on a configuration file (/etc/raidtab) that is difficult to maintain, and partly because its features are limited. In August 2001, Neil Brown released an alternative. His mdadm package provides a simple, yet robust way to manage software arrays.

However, if you find raidtools cumbersome or limited, mdadm (multiple devices admin) is an extremely useful tool for running RAID systems. It can be used as a replacement for the raidtools, or as a supplement.

The main differences between mdadm and raidtools are:

- mdadm can diagnose, monitor and gather detailed information about your arrays
- mdadm is a single centralized program and not a collection of disperse programs, so there's a common syntax for every RAID management command
- mdadm can perform almost all of its functions without having a configuration file and does not use one by default
- Also, if a configuration file is needed, mdadm will help with management of it's contents

Follow these steps to set up software RAID in Linux:

1. configure the RAID driver
2. Initialise the RAID drive
3. Check the replication using /proc/mdstat
4. automate RAID activation after reboot
5. mount the filesystem on the RAID drive

Steps in raid deployment on linux (Continued)

1- Prepare the partition for auto RAID detection using fdisk

In order for a partition to be automatically recognised as part of a RAID set, it must first have its partition type set to fd. This may be achieved by using the fdisk command menu, and using the "t" option to change the setting. Available settings may be listed by using the "l" menu option.

The description may vary accross implementations, but should clearly show the type to be a Linux auto raid. Once set, the settings must be saved back to the partition table using the "w" option.

In working practice, it may be that a physical disk containing the chosen partition for use in a RAID set, also contains partitions for local filesystems, and not intended for inclusion within the RAID set. In order for fdisk to write back the changed partition table, all of the partitions on the physical disk must be not be in use. For this reason, RAID build planning should take into account factors which may not allow the action to be performed truely online.

Steps in raid deployment on linux (Continued)

2 create the array raidset using mdadm

To create an array for the first time, we need to have identified the partitions that will be used to form the RAIDset, and verify with fdisk -l that the fd partition type has been set. Once this has been achieved, the array may be created as follows. mdadm -C /dev/md0 -l raid5 -n 3 /dev/partition1 /dev/partition2 /dev/partition3

This would create, and activate a raid5 array called md0,containing three devices, named /dev/partition1 /dev/partition2, and /dev/partiton3. Once created, and running, the status of the array may be checked by cat'ing /proc/mdstat.

3 Create filesystems on the newly created raidset

The newly created RAID set may then be addressed as a single device using the /dev/md0 device file, and formated, and mounted as normal. For example, mkfs.ext3 /dev/md0.

Steps in raid deployment on linux

4 create the /etc/mdadm.conf using the mdadm command.

Creating the /etc/mdadm.conf file is peasantly simple, requiring only that mdadm be called with the "scan", "verbose", and "detail" switches, and its standard output redirected. It may therefore also be ued as a handy tool for determining any changes on the array simply by diff'ing the current, and stored output. Create as follows

mdadm --detail --scan --verbose > /etc/mdadm.conf

5 Create mount points and edit /etc/fstab

Care must be taken that any recycled partitions used in the RAID set be removed from the /etc/fstab if they still exist.

6 Mount filesystems usingmount

If the array has been created online, and a reboot has not been executed, then the file systems will need to be manually mounted, either manually using he mount command, or simply using **mount -a**

Note:

- **Check the replication using /proc/mdstat.** The /proc/mdstat file shows the state of the kernels RAID/md driver.

- **Automate RAID activation after reboot.** Run **mdadm --assemble** in one of the startup files (e.g. /etc/rc.d/rc.sysinit or /etc/init.d/rcS) When called with the "-s" option (scan) to **mdadm --assemble**, instructs **mdadm** to use the /etc/mdadm.conf if it exists, and otherwise to fallback to /proc/mdstat for missing information. A typical system startup entry could be for example, **mdadm --assemble -s**.

- **mount the filesystem on the RAID drive.** Edit /etc/fstab to automatically mount the filesystem on the RAID drive. The entry in /etc/fstab is identical to normal block devices containing file systems.

- **Manual RAID activation and mounting.** Run **mdadm --assemble** for each RAID block device you want to start manually. After starting the RAID device you can mount any filesystem present on the device using **mount** with the appropriate options.

- **Configuring RAID (alternative)**

  Instead of managing RAID via **mdadm**, it can be configured via /etc/raidtab and controlled via **raidstart**.